

Appendix 1.A Instructional prompts

1.A.1 Concrete Example: RELATEDNESS Prompt

For the entity “Adele” with triples [1. born in London, 2. has genre pop, 3. has label XL Recordings, ...], the instantiated RELATEDNESS prompt is:

Instructional prompt for think component in generation candidates summary for relatedness

You are evaluating triples for RELATEDNESS to the entity.

Entity: Adele

RELATEDNESS DEFINITION:

A triple is RELATED/CENTRAL if it:

1. CENTRALITY: Uses core/frequent predicates that define entity types (identity, classification, basic properties)
2. SPECIFICITY: Provides distinctive values NOT generic descriptions
3. ESSENTIALITY: Best answers "What fundamentally IS this entity?"

DOMAIN CONTEXT - Core/frequent predicates:

birthPlace, genre, recordLabel, birthDate, ...

SELECTION CRITERION: Choose the candidate combining: Centrality (frequent predicate?) + Specificity (distinctive value?) + Essentiality (defines the entity?)

Already selected:

None yet.

Remaining candidates:

1. birthPlace: London
2. genre: pop
3. recordLabel: XL Recordings
4. recordLabel: Columbia
- ...

For each candidate, briefly evaluate: predicate frequency + value specificity + whether it defines the entity.

Select ONE triple index that is MOST RELATED/CENTRAL to the entity.
{exclusion_note}

Output ONLY the integer index:

1.A.2 Concrete Example: INFORMATIVENESS Prompt

For the same entity and selection state, the INFORMATIVENESS prompt guides toward novel, specific information:

Instructional prompt for think component in generation candidates summary for relatedness

You are evaluating triples for INFORMATIVENESS.

Entity: Adele

INFORMATIVENESS DEFINITION:

A triple is informative if it combines:

1. RARITY: Uses uncommon predicates (not generic like `rdf:type`)
2. NOVELTY: Introduces predicates NOT already in your selection
3. SPECIFICITY: Concrete, detailed values (not generic categories)

DOMAIN CONTEXT - Rare predicates in dataset:

grammy_awards, writing_credits, collaborations, ...

SELECTION CRITERION:

Choose the candidate with highest information gain:

- PredicateRarity (uncommon?) + TopicNovelty (covers new?)
+ Specificity

CURRENT STATE - Already selected predicates:

birthPlace, genre

Remaining candidates:

3. recordLabel: XL Recordings
5. grammy_awards: 15
6. writing_credits: Someone Like You

Select ONE triple index that is MOST INFORMATIVE
(combines rarity + novelty + specificity).

Output ONLY the integer index:

1.A.3 Concrete Example: COVERAGE/DIVERSITY Prompt

For ensuring diverse semantic coverage, the COVERAGE prompt tracks which facets have been addressed:

Instructional prompt for think component in generation candidates summary for relatedness

You are evaluating triples for DIVERSITY and coverage.

Entity: Adele

DIVERSITY DEFINITION

A triple MAXIMIZES DIVERSITY if it:

1. ROLE VARIETY: Covers semantic roles NOT yet represented
2. PREDICATE NOVELTY: Uses different predicates from selected
3. PERSPECTIVE BREADTH: Views entity from different aspects

CURRENT STATE - Semantic roles covered:

location(1), genre(1)

AVAILABLE OPPORTUNITIES - Roles NOT yet covered:

awards, collaborations, achievements, timeline, ...

SELECTION CRITERION:

Choose the candidate with highest diversity impact:

- RoleNovelty (covers missing semantic role?)
- + PredicateNovelty (different predicate?)
- + PerspectiveBreadth (new aspect?)

Already selected:

1. birthPlace: London
2. genre: pop

Remaining candidates:

5. grammy_awards: 15 [awards role]
6. collaborations: Beyonce [collaboration role]
7. album_release: 21 [temporal role]
- ...

Select ONE triple that MAXIMIZES DIVERSITY and coverage.

Output ONLY the integer index:

Appendix 1.B ToT4ES Algorithm

Algorithm 1: ToT4ES: Tree-of-Thoughts Entity Summarization

Input:
 Global description $\text{Desc}(e, \mathcal{T})$ (an entity description of the entity)
 Triple set $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$
 Maximum summary length k
 Beam width β
 LLM model $\mathcal{M}$
 Temperature parameters: $\text{Temperature}_{\text{thought}}$, $\text{Temperature}_{\text{eval}}$

Output: Summary: $S^* = [s_1, s_2, \dots, s_k]$ (selected triples)

```

1 root  $\leftarrow$  TreeNode(state =  $\emptyset$ );
2 queue  $\leftarrow$  [root];
3 for step  $\leftarrow$  1 to  $k$  do
4   layer_size  $\leftarrow$  |queue|;
5   children  $\leftarrow$   $\emptyset$ ;
6   for  $i \leftarrow$  1 to layer_size do
7     node  $\leftarrow$  queue.popleft();
8      $s_{\text{current}} \leftarrow$  node.state;
9     /* Generate candidate thoughts via thought policy */
10     $\text{Desc}(e, \mathcal{T}) \leftarrow$  ThoughtGen( $\mathcal{D}, s_{\text{current}}, \mathcal{M}, \text{Temperature}_{\text{thought}}$ );
11     $\mathcal{C} \leftarrow$  Deduplicate( $\mathcal{C}, s_{\text{current}}$ );
12    foreach  $c \in \mathcal{C}$  do
13       $s_{\text{new}} \leftarrow s_{\text{current}} \oplus c$ ; // Append triple to state
14      children.add(TreeNode( $s_{\text{new}}, \text{thought} = c$ ));
15  if |children| = 0 then
16    break;
17  /* Evaluate all children via state evaluator */
18  states  $\leftarrow$  [node.state | node  $\in$  children];
19  scores  $\leftarrow$  StateEval( $\text{Desc}(e, \mathcal{T}), \text{states}, \mathcal{M}, \text{Temperature}_{\text{eval}}$ );
20  /* Prune to beam width */
21  ranked  $\leftarrow$  Sort(children by scores, descending);
22  queue  $\leftarrow$  ranked[1 : min( $\beta$ , |ranked|)];
23  $S^* \leftarrow$  queue[1].state ; // Return best final state

```
